CS 303

FRONTEND

FRAMEWORKS

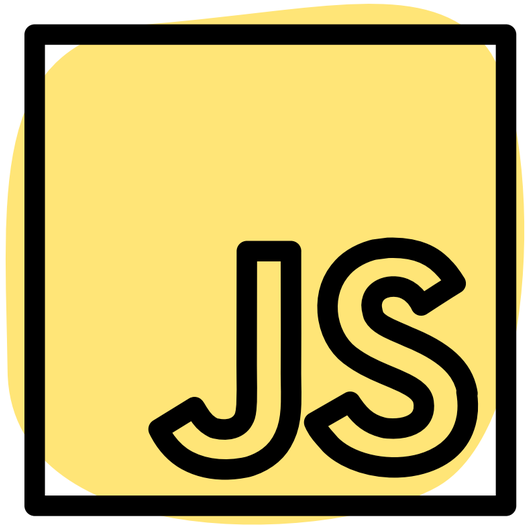
(CONTINUED)

JAVASCRIPT IN THE FE

Purpose: to make web pages dynamic

Examples:

- Event listeners
- DOM manipulation
- Data fetching



EVENT LISTENERS

3 ways to add event listeners in vanilla js

1 way to remove

Types:

- click
- mouseover
- drag
- etc...

```
<button id="button"
  onclick="console.log('clicked from html')">
  Click me!
</button>
```

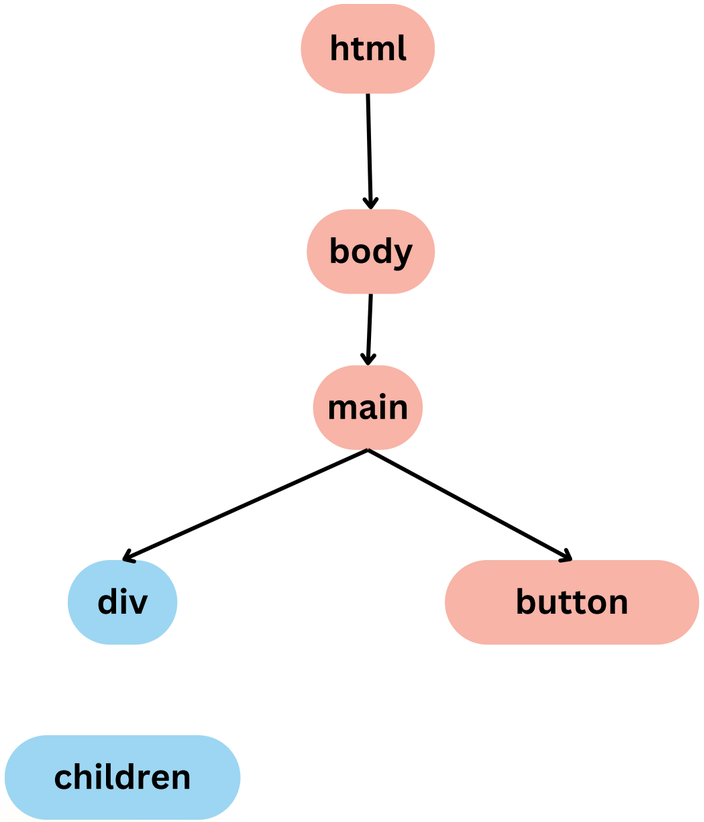
```
const button = document.querySelector("#button");

button.addEventListener("click", () => {
  console.log('add event listener click');
});
```

```
const button = document.querySelector("#button");
button.onclick = function() {
  console.log('clicked');
}
```

```
button.removeEventListener("click", buttonClickHandler);
```

EVENT PROPAGATION

Events will move up the DOM tree

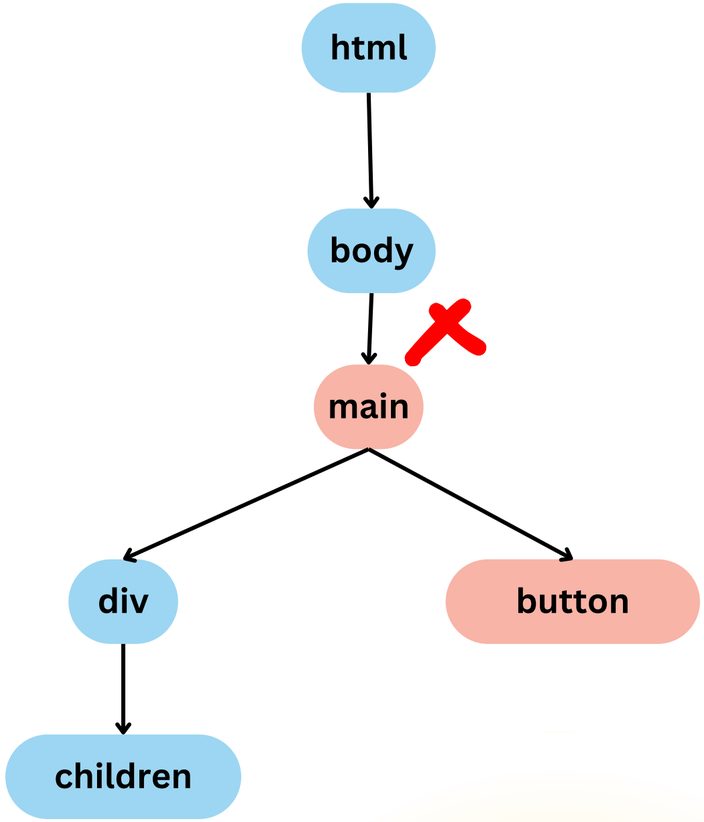


click fired

continues bubbling up the dom tree to trigger other handlers for click event

PREVENTING PROPAGATION

Use preventDefault() or stopPropagation()




```
main.addEventListener("click", (e) => {
  e.stopPropagation();
  console.log("main click handler");
});
```

stackblitz.com/edit/cs303-js-todo

WHAT IS A FRAMEWORK?

Framework = scaffolding for building out a software system

- **Opinionated** – heavy lifting on how things should be rendered have been made
- **Declarative** – reduces boilerplate
- **Quicker ramp up** – expectations on how framework works is same regardless of which project you join
    - No bespoke solutions

4 COMMONALITIES

1. Components
2. State Handling
3. Special rendering (ex: VDOM)
4. Dependency Injection

4 COMMONALITIES

1. **Components**
2. State Handling
3. Special rendering (ex: VDOM)
4. Dependency Injection

COMPONENTS

What are components?
Reusable pieces of code that serve as UI building blocks

- Makes code maintainable

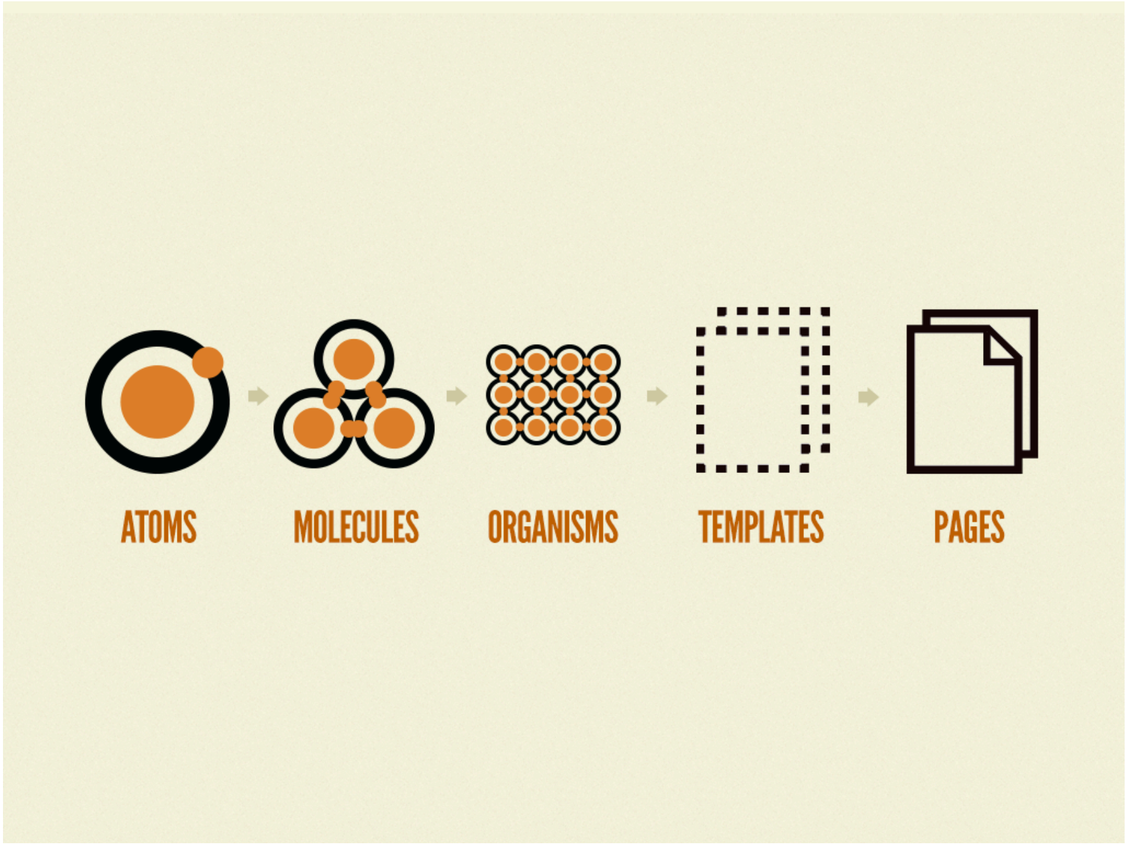
```jsx
import React from 'react';

export default function MyComponent() {
  return <div>Hello, this is my React component!</div>;
}
```

COMPONENTS

Components Thinking:

- Breaking down the UI into the smallest usable unit and then building from here

Composability: – components can be made from other components



COMPONENTS

What are components?
Reusable pieces of code that serve as UI building blocks

- Makes code maintainable

Components Thinking:

- Breaking down the UI into the smallest usable unit and then building from here

Composability: – components can be made from other components

```jsx
import React from 'react';

export default function MyComponent() {
  return <div>Hello, this is my React component!</div>;
}
```

Your turn: With your partner, create a list of components from the to-do list

- Don’t forget to capture this in your learning doc

stackblitz.com/edit/cs303-js-todo

4 COMMONALITIES

1. Components
2. **State Handling**
3. Special rendering (ex: VDOM)
4. Dependency Injection

STATE HANDLING IN THE FE

State = the user interface based on the user interaction

Ex: toggling light/dark mode on a web app

- **State**: mode <--- holds ‘light’ or ‘dark’
- **Effect:** applies either light or dark theme

```
export default function MyComponent() {
  const [myState, setState] = useState('My state here ✨');
  return (
    <div>
      Hello, this is my React component!
      {myState}
    </div>
  );
}
```

STATE HANDLING IN THE FE

State = the user interface based on the user interaction

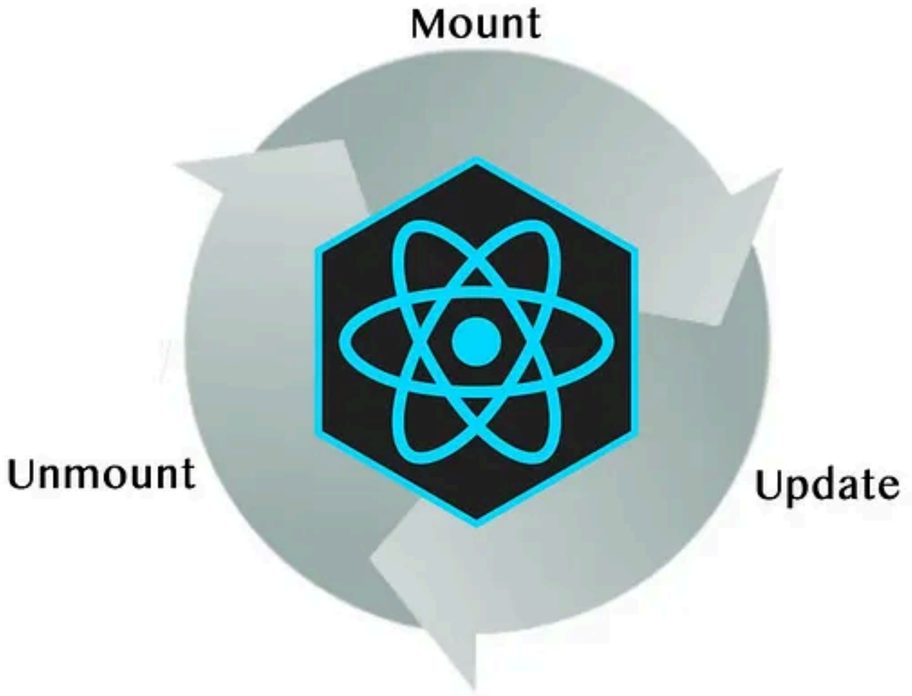
Your turn:

- With your partner – list the states that should be tracked for the TodoItem component
  - Capture this in your learning doc by linking to your forked stackblitz

stackblitz.com/edit/cs303-react-todos

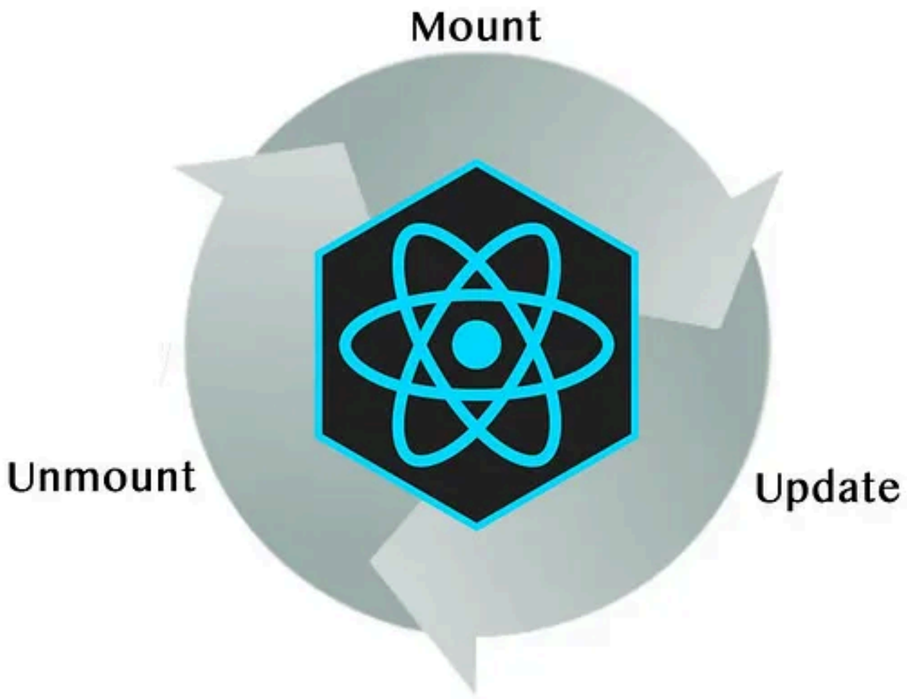
STATE HANDLING IN THE FE

Component Lifecycle



STATE HANDLING IN THE FE

Component Lifecycle



Trigger render

2 reasons to do this

1. Component’s initial render
2. Component’s state (or ancestor) has been updated

Render component

Commit to DOM

COMPONENTS + STATE

Put the 2 previous concepts together and refactor the <form> in the React Todo app into its own component

- Have more time? Try refactoring other components from your list
- Reminder: Link to your forked stackblitz in your learning doc **stackblitz.com/edit/cs303-react-todos**

PROJECT #1 OVERVIEW

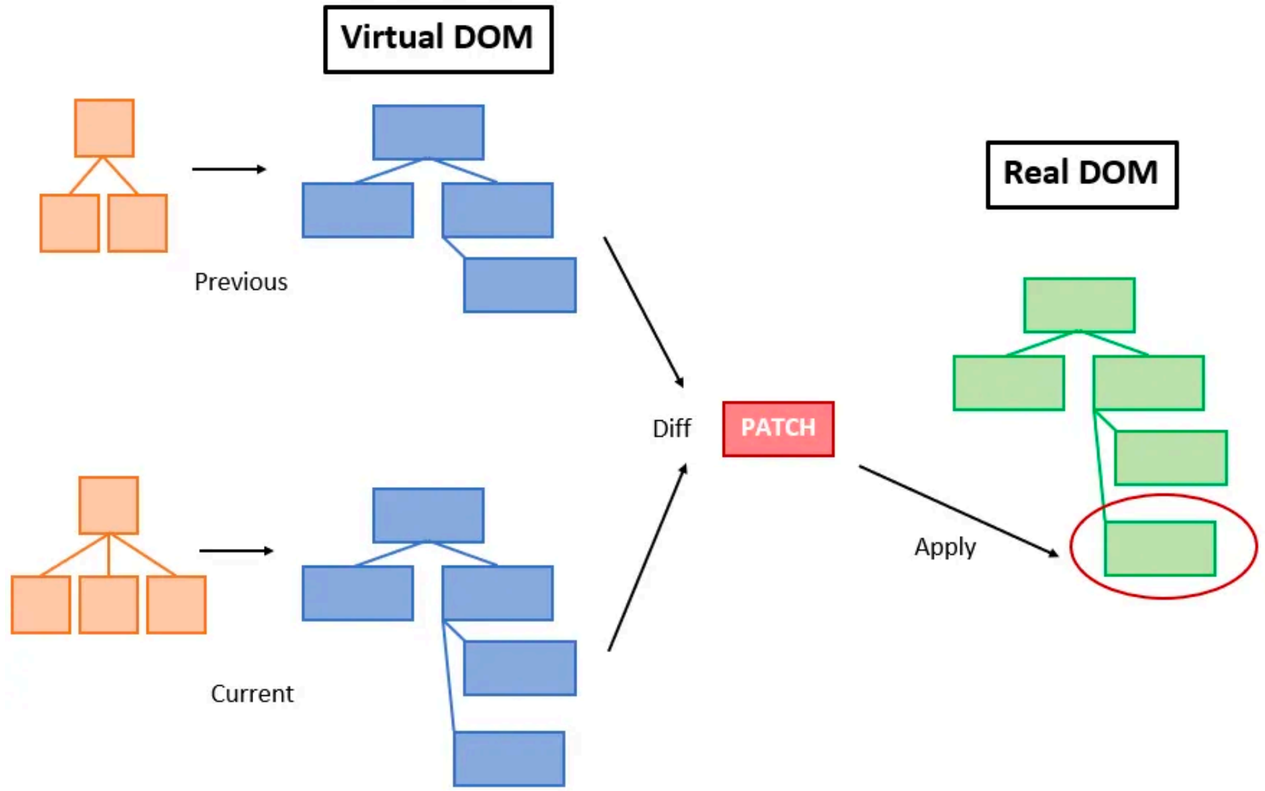
Project Spec

4 COMMONALITIES

1. Components
2. State Handling
3. **Special rendering (ex: VDOM)**
4. Dependency Injection

VIRTUAL DOM (VDOM)

1. Virtual DOM
2. When changes are made:
3. Rerender a new virtual DOM tree
4. Diff against previous virtual DOM tree
5. Make changes to the actual (physical) DOM



REFLECTION

For your learning doc:

1. In your own words – what is the Virtual DOM and why is it more efficient than manipulating the physical browser DOM?
2. What other aspects of FE frameworks do you want to learn?

4 COMMONALITIES

1. Components
2. State Handling
3. Special rendering (ex: VDOM)
4. **Dependency Injection (for next time)**